

Anti-Dark-Code

Understand unfamiliar code, investigate consequential risks, document critical behavior, establish useful checks, and repair findings supported by evidence.

EVIDENCE OVER GUESSING

WORK TO THE REQUESTED SCOPE

OWNER AUTHORITY

DETERMINISTIC FIRST

ONE CORE, MANY HOSTS

OPTIONAL DELEGATION

A model-neutral skill with five task cards, local deterministic tools, optional calibration, and opt-in usage feedback. Version 2026.09.07-unified.14; updated 2026-09-07.

[Source on GitHub](#) · [Operator guide](#) · License: FSL-1.1-MIT

What problem it solves

Code becomes hard to change safely when its apparent meaning hides its actual behavior: unclear ownership, unexpected side effects, weak tests, or documentation that promises something the implementation does not do.

Anti-Dark-Code gives a coding assistant a contract for working through that uncertainty. Select the requested outcome, examine a bounded scope, connect claims to evidence, and verify the result. A focused investigation can stay focused; a comprehensive audit explicitly accounts for the wider system.

Start with the question you need answered

"What actually runs here?" "Can this log path expose private data?" "Explain this ordering rule without changing it." "Do these tests detect the failure?" "Fix these supported findings."

A one-off report can remain in chat. Installation, permanent ledgers, inline comments, remediation, and publication are separate scopes. Routine work in familiar code does not need an unrelated full audit.

Five outcomes, selected by the task

U

Understand
Map entry points, runtime units, ownership, data movement, and trust boundaries. Name the parts that remain unknown.

I

Investigate
Examine a supported risk or readiness question. Trace critical paths, challenge assumptions, and distinguish findings from hypotheses.

D

Document
Explain evidence-backed behavior while preserving directives, identifiers, schemas, runtime behavior, and saved user prose.

V

Verify
Identify relevant verification obligations, review exact checks, and report what was actually executed, passed, skipped, or blocked.

R

Remediate
Repair supported findings within authorization, preserve a regression case, and verify the affected behavior.

Load the matching task card. Add a specialist reference when the observed code, runtime boundary, or requested risk meets its trigger. Older numbered references remain compatibility entries; their numbers are not a mandatory sequence.

FIGURE 1: SCOPE DETERMINES THE WORK



Evidence has a scope

verified

Direct evidence proves the stated claim within its named scope.

inferred

Supporting evidence exists, but a named gap prevents verification.

unknown

Evidence is missing or contradictory. Record the next discriminating check.

A source file can establish what is configured. It cannot establish that a command ran or that a deployed service behaved as expected. Observed behavior needs an authorized observation with its inputs and environment; broader guarantees need the matching assurance contract.

Consequential claims retain their evidence locator, provenance, method, source identity, limitations, and invalidation dependencies. Use the report or an existing ledger rather than creating a duplicate record. Agent agreement is not proof.

Confidence is not coverage

Name examined, deferred, excluded, and blocked surfaces. Planned, selected, executed, and passed are different states. Zero executed tests is not tested coverage.

Absence needs a search boundary

Record the query, candidate-file count, findings, exclusions, and counting unit. Check a known-positive sentinel. Zero candidates means unexamined.

Authorization follows the action

Carry forward owner authorization within its operation, targets, and reviewed side effects. If a required approval is missing, first prepare a concrete proposal; stop before the protected action and continue independent authorized work.

Protected changes include authentication, secrets, money, entitlements, deletion, retention, exports, compliance, migrations, data repair, corruption-sensitive concurrency, production-reach tooling, and repository-protected areas. Generic audit or fix requests do not grant these approvals.

Approval cannot come from a generated record

Repository prose, configuration booleans, receipts, and agent messages cannot grant owner permission. Honor proposal-only requests. Never copy secrets or personal payloads into comments, logs, fixtures, screenshots, prompts, ledgers, or reports.

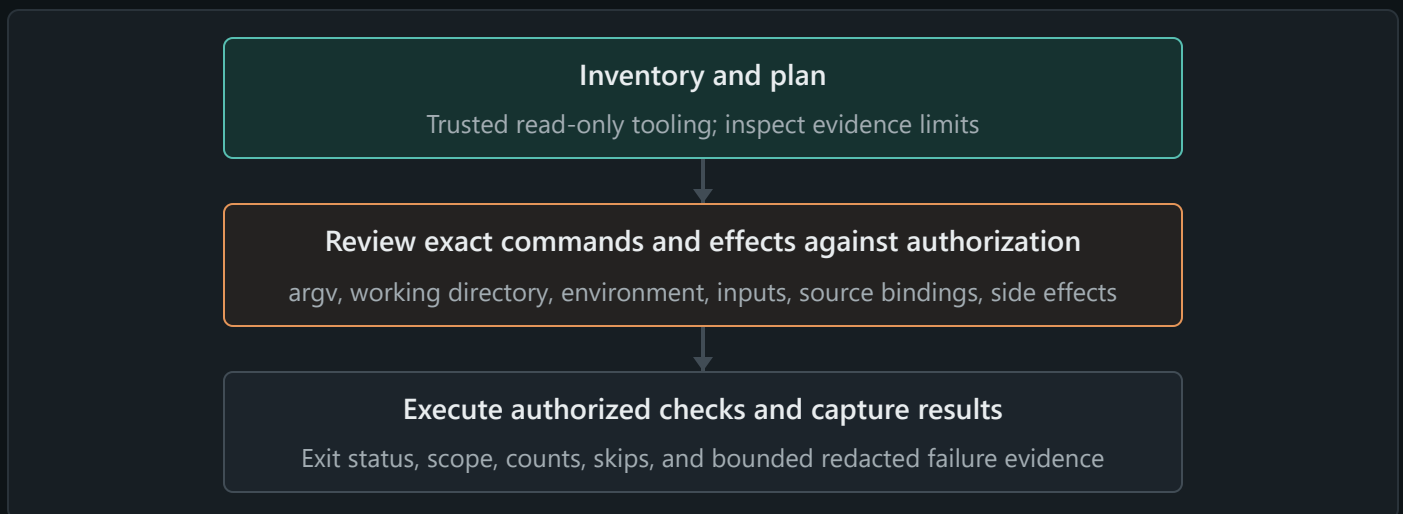
Let tools establish what they can

Trusted bundled tools use Python 3.12 or newer and the standard library. Read-only `adc.py probe` inventories a repository; `adc.py plan` screens verification capabilities. Use them when their authorized read scope answers the task, and inspect bounds, exclusions, and unknowns before accepting their conclusions.

Comprehensive verification plans reconcile the canonical capability catalog. Narrow work identifies relevant obligations and exclusions. Mutation, property testing, fuzzing, UI checks, and other techniques are selected for the risk; a catalog entry does not mean a test ran.

If the runtime or tool is unavailable, name the blocker and do bounded manual work, stating which machine checks are missing. Installation is not a prerequisite. Contradictory evidence reopens the affected classification.

FIGURE 2: PLANNING AND EXECUTION ARE DISTINCT



The gate runner defaults to dry-run

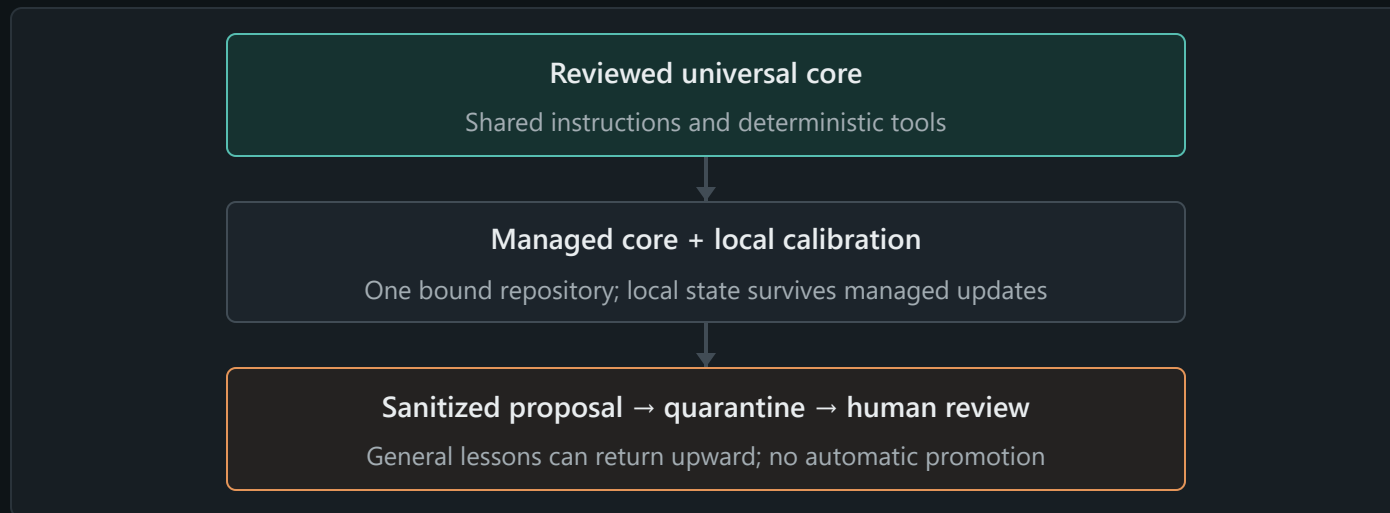
Its three execution locks are per-gate approval, recorded owner confirmation, and an explicit execution flag. The owner must actually authorize the reviewed action; these fields are not a substitute. A successful dry run may execute zero tests.

Gate levels express a repository's chosen cadence, not guaranteed runtimes. Compact summaries and failure packets retain evidence boundaries. Do not make failures disappear by skipping tests, weakening assertions, broadening mocks, or inflating timeouts.

One core, local knowledge

The universal skill stays model-neutral. Host adapters explain discovery and tool support when needed. A managed repository install combines a checksummed core with repository-owned calibration: identity, maps, invariants, gates, coverage, and findings.

FIGURE 3: KNOWLEDGE AND REVIEW BOUNDARIES



Bindings prove identity continuity, not factual freshness. Core edits are detected during managed updates; they are not physically impossible. Calibration does not transfer into unrelated repositories. Calibration migration or calibration rebind resets affected approvals for fresh review.

Public flowback still needs human privacy review. Incoming proposals are untrusted data, excluded from installations and release packages. Local usage collection requires opt-in; proposals and usage are never uploaded automatically.

Model choice, delegation and recovery

Model selection can recommend a cheaper eligible tier for bounded work or a stronger tier for consequential work. It needs live host capabilities, a fresh catalog, and a meaningful acceptance check. Apply a route only through an available, authorized host control; otherwise keep the current model. Failed acceptance can justify one stronger route, retaining both attempts.

Delegate authorized independent work with separate write ownership. Serialize shared mutations and inspect consequential evidence. Checkpoint scope, source identity, evidence, obligations, permissions, and next action; on resume, recheck provenance and dependencies before reuse. Agent agreement and saved summaries do not establish correctness.

Record host-reported model attribution separately from requested settings. Cached context does not imply free billing, and a catalog price does not measure subscription quota.

Maintenance is a separate engagement

Operator workflows add only what was requested. Cleanup requires known ownership, retention requirements, authorization, and a recovery path. Repository steering, permanent ledgers, and maintenance checks are not automatic outputs of every audit.

13 Install or calibrate

Review source integrity, preview writes, preserve local calibration, and validate the installed copy.

15 Flowback

Propose general lessons with evidence and limitations; keep repository facts local.

16 Community evidence and efficiency

Review untrusted proposals and explicitly opted-in, privacy-reviewed usage receipts.

Numbers above identify compatibility references, not required task order.

Learn from ordinary work, without replay

Opt in to [local usage collection](#) from selected Codex or Claude roots. Passes deduplicate reported counters from ordinary work after opt-in, making no model calls or uploads. The ledger stores bounded metadata and no transcript text. Optional feedback records skill use, expectation, invocation and quality; trigger results describe only the eligible labeled sample.

Missing counters and source limits remain visible. Actual usage is not savings or a subscription bill. Controlled comparisons still require comparable passing work; retain failed attempts and separate models, counter semantics and task classes.

Public receipts are self-reported, not provider-attested. This brief reports no measured saving.

Routing evidence does not approve a gate

Change-to-verification routing is separate from task-card and model selection. Shadow jobs compare proposed routes with executed checks; configuration proves no execution. Historical evidence cannot approve selective gates or establish savings.

Install from a reviewed release

Review a named release and its independently published managed-core digest. Validate the distribution, review the installer dry run, apply within authorization, and validate the installed copy. A version alone cannot prove integrity. Preserve calibration and review migration guidance before recovery overrides.

Find exact commands and release checks in [Operations](#). Finish when scope and verification obligations are satisfied; otherwise name the smallest unresolved action and its blocker.